

Sponsored Placements Design

Status: Proposed **Created:** 2026-02-28 **Related:** [specs/payout-design.md](#), [specs/retailer-subscriptions-design.md](#), [specs/affiliate-network-design.md](#)

Overview

Sponsored placements allow retailers to pay LYKE to promote specific products within the shopper feed, targeting users by body profile and category affinity. This document covers the billing model, campaign lifecycle, ad serving logic, creator fee attribution, and all data model and API changes required.

Current State

The [SponsoredPlacement](#) entity and campaign management endpoints are fully built:

What exists	Gap
SponsoredPlacement entity with BudgetAmount and SpentAmount	SpentAmount never increments — no serving logic
POST /api/retailers/v1/portal/campaigns — create campaign	No payment step; campaigns activate immediately for free
GET /api/retailers/v1/portal/campaigns — list campaigns	Computed status from date/active flag only
PUT /api/retailers/v1/portal/campaigns/{id} — update campaign	No pause/cancel lifecycle states
Body type and category targeting stored as JSON	Never used at serve time
RetailerService computes campaign status from dates	No CampaignStatus field on entity

Pricing Model

MVP: Fixed fee per campaign.

The retailer agrees a fixed spend for a campaign over a defined date range. The full amount is charged at campaign activation. No per-impression or per-click billing. This is the simplest model to implement, easy to sell to the first retailer, and requires no real-time impression counting infrastructure.

Future phase	Description
CPM (Phase 2)	Cost per 1,000 impressions served. Requires impression tracking and a spend-capping job.
CPC (Phase 2)	Cost per click on sponsored product. Uses existing ClickEvent data.

Indicative pricing for fixed-fee campaigns (LYKE sets rates commercially, not in code):

Placement type	Indicative rate
Body-profile matched product boost	£500–£2,000 / 30 days
Category takeover (all users in a category)	£1,500–£5,000 / 30 days
Creator-linked sponsored post	Negotiated directly (creator fee + LYKE margin)

Retailer Billing Infrastructure

Both sponsored placements and data subscriptions (see [specs/retailer-subscriptions-design.md](#)) bill through a single **Stripe Customer** per retailer. This entity is created once and shared across both billing systems.

New Entity: **RetailerBillingAccount**

```

RetailerBillingAccount
├─ Id (GUID)
├─ RetailerId (FK → Retailer, unique)
├─ StripeCustomerId (string, encrypted)      ← e.g. "cus_ABC123"
├─ DefaultPaymentMethodId (string?, encrypted) ← stored Stripe PM for
repeat charges
├─ Currency (string)                        ← ISO 4217, e.g. "GBP"
├─ CreatedAt (DateTime)
└─ UpdatedAt (DateTime?)

```

StripeCustomerId and **DefaultPaymentMethodId** are encrypted at rest using the same **IEncryptionService** defined in [specs/payout-design.md](#).

A **RetailerBillingAccount** is created lazily — when the retailer first adds a payment method (either for a campaign or a subscription). Creation calls **StripeClient.Customers.CreateAsync** with the retailer's name and contact email.

Campaign Lifecycle

Status Machine

Add a **CampaignStatus** enum field to **SponsoredPlacement**, replacing the current computed string status in **RetailerService.GetCampaignStatus()**.

```

Draft
├─ Retailer submits campaign for payment
▼
AwaitingPayment
├─ Stripe PaymentIntent confirmed
▼

```

```

Active ←—— Resume
|   Retailer pauses / budget exhausted
▼
Paused
|   EndDate reached / admin action
▼
Completed

(from any state before Active)
▼
Cancelled ← Stripe refund issued if AwaitingPayment/Active with unspent
budget

```

New enum: `CampaignStatus`

```

public enum CampaignStatus
{
    Draft           = 0,
    AwaitingPayment = 1,
    Active          = 2,
    Paused          = 3,
    Completed       = 4,
    Cancelled       = 5
}

```

Payment Flow

1. Retailer creates campaign → POST `/api/retailers/v1/portal/campaigns`
 └ CampaignStatus = Draft; no Stripe call yet
2. Retailer confirms campaign → POST `/api/retailers/v1/portal/campaigns/{id}/checkout`
 └ LYKE creates Stripe PaymentIntent for BudgetAmount
 └ Returns { clientSecret, paymentIntentId }
 └ CampaignStatus = AwaitingPayment
3. Frontend completes payment (Stripe.js / Capacitor in-app payment sheet)
 └ Stripe confirms payment
4. Stripe sends `payment_intent.succeeded` webhook → POST `/api/webhooks/stripe`
 └ CampaignService marks CampaignStatus = Active
 └ Sends confirmation email to retailer
 └ Campaign now eligible for serving from StartDate
5. Campaign serves impressions until EndDate or Paused/Cancelled

For fixed-fee campaigns, **SpentAmount** is set to **BudgetAmount** when the campaign becomes **Active** — the budget is fully committed at payment. **SpentAmount** will track actual impression delivery for CPM campaigns in a future phase.

Refund Policy

- **Draft** → **Cancelled**: no charge; PaymentIntent cancelled.
- **AwaitingPayment** → **Cancelled**: PaymentIntent cancelled; no charge.
- **Active** → **Cancelled before StartDate**: full refund via **StripeClient.Refunds.CreateAsync**.
- **Active** → **Cancelled after StartDate**: no refund (campaign has been live). Admin can issue manual partial refunds at their discretion.
- **Budget exhausted**: campaign moves to **Completed**; no refund.

Data Model Changes

Modified: **SponsoredPlacement**

```
SponsoredPlacement
├─ ... (existing fields)
├─ CampaignStatus (enum)           ← replaces computed string; default
Draft
├─ PricingModel (enum)             ← FixedFee | CPM | CPC; MVP =
FixedFee only
├─ CreatorId (Guid?, FK → Creator) ← from payout-design.md; null =
platform-placed
├─ CreatorFeeAmount (decimal?)     ← portion of BudgetAmount paid to
creator
├─ ImpressionCount (int)           ← incremented by serving logic
```

New Entity: **CampaignPayment**

```
CampaignPayment
├─ Id (GUID)
├─ CampaignId (FK → SponsoredPlacement)
├─ RetailerId (FK → Retailer)
├─ StripePaymentIntentId (string, encrypted)
├─ Amount (decimal)
├─ Currency (string)
├─ Status (enum: Pending | Succeeded | Failed | Refunded)
├─ CreatedAt (DateTime)
├─ PaidAt (DateTime?)
```

New Enum: **PricingModel**

```
public enum PricingModel
{
    FixedFee = 0,
    Cpm      = 1,
    Cpc      = 2
}
```

Ad Serving Logic

Placement Strategy

Sponsored content is injected at regular intervals within the shopper's personalised feed. The current `FeedService` returns a ranked list of posts — sponsored posts are inserted as a separate pass after organic ranking.

Injection rule (configurable): Every `N`th position in the feed is a candidate sponsored slot. Default: every 5th post. If no eligible campaign exists for the slot, the next organic post fills the gap — sponsored slots are never left empty or shown as blanks.

```
Position 1:  organic
Position 2:  organic
Position 3:  organic
Position 4:  organic
Position 5:  ← sponsored slot (if eligible campaign exists)
Position 6:  organic
...
Position 10: ← sponsored slot
```

Campaign Eligibility for a Shopper

A campaign is eligible to fill a slot if all of the following are true:

1. `CampaignStatus == Active`
2. `DateTime.UtcNow >= StartDate && DateTime.UtcNow <= EndDate`
3. `TargetBodyTypes` is null **or** contains the shopper's body type
4. `TargetCategories` is null **or** overlaps with the shopper's browsed/saved categories
5. For CPM (future): `SpentAmount < BudgetAmount`

When multiple campaigns are eligible, selection priority:

1. Campaigns linked to a creator whose posts the shopper has engaged with (highest relevance)
2. Campaigns with the most specific targeting (non-null body type AND category)
3. Campaigns with the most remaining duration (avoids exhausting long campaigns early)

Impression Tracking

For fixed-fee campaigns, impressions are recorded for reporting only. Each time a sponsored slot is filled:

- Increment `SponsoredPlacement.ImpressionCount`
- Fire an `AnalyticsEventType.SponsoredImpression` event (App Insights) with `campaignId` and `shopperId` (hashed)

Clicks on sponsored products already flow through the existing `POST /api/clicks/track` endpoint. Tag sponsored clicks by including `campaignId` in `ClickEvent.AttributionData`.

Feed Service Changes

`FeedService` requires two changes:

1. A new `GetActiveCampaignsForShopper(shopperBodyProfile)` query that returns eligible campaigns sorted by priority.
2. A `InjectSponsoredSlots(posts, campaigns, interval)` method that merges the two lists using the injection rule.

The sponsored post returned to the frontend uses the same `PostResponse` DTO with an added `IsSponsored: true` flag and `CampaignId`. The frontend renders a "Sponsored" badge.

Creator Fee Attribution

When a campaign has a `CreatorId` set, a portion of the campaign fee is paid to the creator:

- `CreatorFeeAmount` is agreed at campaign setup and stored on `SponsoredPlacement`
- When the campaign transitions to `Active`, a `CreatorEarning` of `EarningType.Sponsored` is created for `CreatorFeeAmount` with `Status = Confirmed` immediately (no attribution window — it's a flat fee, not performance-based)
- LYKE retains `BudgetAmount - CreatorFeeAmount`
- The creator's `Confirmed` earning enters the normal payout flow described in `specs/payout-design.md`

If `CreatorId` is null (platform-placed campaign), LYKE retains 100% of `BudgetAmount`.

New API Endpoints

All new endpoints added to `RetailerEndpoints.cs` under the existing `/api/retailers/v1/portal/` prefix, except the webhook which is handled in `WebhookEndpoints.cs`.

```
POST    /api/retailers/v1/portal/campaigns/{id}/checkout
→ Create Stripe PaymentIntent for campaign budget
→ Returns { clientSecret, paymentIntentId, amount, currency }
→ Requires CampaignStatus == Draft

POST    /api/retailers/v1/portal/campaigns/{id}/pause
→ Set CampaignStatus = Paused
→ Requires CampaignStatus == Active

POST    /api/retailers/v1/portal/campaigns/{id}/resume
→ Set CampaignStatus = Active
```

→ Requires CampaignStatus == Paused and EndDate > now

POST /api/retailers/v1/portal/campaigns/{id}/cancel

→ Set CampaignStatus = Cancelled

→ Issue Stripe refund if applicable (see refund policy above)

GET /api/retailers/v1/portal/billing/setup

→ Create or retrieve RetailerBillingAccount

→ Returns Stripe SetupIntent client secret (for saving a payment method)

GET /api/retailers/v1/portal/campaigns/{id}/payment

→ Get CampaignPayment status for a campaign

POST /api/webhooks/stripe (extend existing handler)

→ Handle payment_intent.succeeded → activate campaign

→ Handle payment_intent.payment_failed → notify retailer, revert to Draft

→ Handle charge.refunded → update CampaignPayment status

New Services

ISponsoredCampaignService (Application layer)

```
public interface ISponsoredCampaignService
{
    Task<CheckoutResponse> CreateCheckoutAsync(Guid retailerId, Guid
campaignId, CancellationToken ct = default);
    Task PauseCampaignAsync(Guid retailerId, Guid campaignId,
CancellationToken ct = default);
    Task ResumeCampaignAsync(Guid retailerId, Guid campaignId,
CancellationToken ct = default);
    Task CancelCampaignAsync(Guid retailerId, Guid campaignId,
CancellationToken ct = default);
    Task HandlePaymentSucceededAsync(string paymentIntentId,
CancellationToken ct = default);
    Task HandlePaymentFailedAsync(string paymentIntentId, CancellationToken
ct = default);
    Task<IReadOnlyList<SponsoredPlacement>>
GetEligibleCampaignsAsync(ShopperProfile profile, CancellationToken ct =
default);
}
```

IRetailerBillingService (Application layer, shared with subscriptions)

```
public interface IRetailerBillingService
{
    Task<RetailerBillingAccount> GetOrCreateBillingAccountAsync(Guid
retailerId, CancellationToken ct = default);
    Task<string> CreateSetupIntentAsync(Guid retailerId, CancellationToken
```

```
ct = default);
    Task SaveDefaultPaymentMethodAsync(Guid retailerId, string
stripePaymentMethodId, CancellationToken ct = default);
}
```

Configuration

SponsoredPlacementSettings

```
public class SponsoredPlacementSettings
{
    public const string SectionName = "SponsoredPlacement";

    // Feed injection interval: 1 in N posts is a sponsored slot
    public int FeedInjectionInterval { get; set; } = 5;

    // Minimum campaign budget (validated on CreateCampaign)
    public decimal MinBudgetAmount { get; set; } = 250.00m;

    // Maximum campaign duration in days
    public int MaxCampaignDurationDays { get; set; } = 90;
}
```

Email Notifications

Event	Template	Recipient
Campaign payment confirmed	campaign-activated.liquid	Retailer
Campaign payment failed	campaign-payment-failed.liquid	Retailer
Campaign starts (on StartDate)	campaign-started.liquid	Retailer
Campaign ends (on EndDate)	campaign-completed.liquid	Retailer + performance summary
Campaign cancelled + refund issued	campaign-cancelled.liquid	Retailer
Creator fee credited	sponsored-earning-credited.liquid	Creator

Frontend Changes

Retailer — Campaign Create/Edit (update existing)

- Add payment step to the existing campaign creation wizard (new final step: payment)
- Show Stripe payment form (Elements) on checkout step
- On successful payment, navigate to campaign detail with "Campaign activated" confirmation

Retailer — Campaign List (update existing)

- Show new **CampaignStatus** values with appropriate badges and actions:
 - Draft → "Pay to activate" button
 - AwaitingPayment → spinner / "Processing payment"
 - Active → "Pause" button + impression count
 - Paused → "Resume" button
 - Completed → "View report" button
 - Cancelled → read-only

Retailer — Billing Setup (new page)

- Route: **/retailer/billing**
- Add payment method using Stripe Elements (SetupIntent flow)
- Show saved payment method (last 4 digits, expiry)
- Link to Stripe billing portal for full history

Shopper — Feed (update existing)

- Render "Sponsored" badge on injected sponsored posts
- Sponsored posts use the existing **PostCard** component with an added badge input

Open Questions

1. **Admin approval gate for campaigns** — should LYKE review campaigns before they go live? For MVP with a trusted single retailer this is not needed. Add an **AdminApprovalRequired** flag to **SponsoredPlacementSettings** for future enforcement.
2. **Frequency capping** — should a shopper see the same campaign more than once per session/day? For MVP: no cap (keep it simple). Add **MaxImpressionsPerUserPerDay** to **SponsoredPlacementSettings** in Phase 2.
3. **CPM transition** — when the CPM model is introduced, **SpentAmount** will need to increment in real time. A background job or queue-based approach will be needed to avoid race conditions on high-traffic feeds.
4. **Campaign performance reporting** — the campaign completed email references a "performance summary". This needs a **CampaignAnalytics** query (impressions, clicks, conversion rate) added to **RetailerService**. Defer to Phase 2 alongside CPM.